

S8 INTEGRATED PROJECT • 2025–2026

Product Review Intelligence

*A Multi-Agent AI System for Customer Insight
and Competitive Analysis*

Major: Computer Engineering — Big Data & Artificial Intelligence

By:

Chaymae Benmakhlouf

Maroua Ait Sassi

Rania Bouaroua

Supervised by Prof. Hakim Hafidi

2025 – 2026

Table of Contents

1. Executive Summary-----	3
2. Problem and Motivation -----	3
3. System Design and Architecture -----	4
3.1 Why multi-agent-----	4
3.2 The three agents -----	5
3.3 Data flow and communication -----	5
3.4 Tools — input / output schemas -----	6
4. Deep Learning Model -----	6
4.1 Dataset and preprocessing-----	6
4.2 Architecture and training setup -----	7
4.3 Evaluation results -----	7
5. The Multi-Agent System in Action -----	8
5.1 End-to-end run-----	8
5.2 The Streamlit interface -----	9
5.3 Human-in-the-loop checkpoint -----	9
6. Evaluation and Robustness -----	9
7. Reproducibility and Engineering -----	10
8. Limitations and Future Work-----	11
9. Reflection -----	11
References-----	12

List of Figures

Figure 1 — Communication diagram (User → Orchestrator → Agents → HITL)-----	5
Figure 2 — Confusion matrix on 5,000 held-out test examples -----	7
Figure 3 — Streamlit landing page-----	8
Figure 4 — Pipeline running pane-----	8
Figure 5 — Generated brief with HITL controls-----	9
Figure 6 — JSONL action log in terminal -----	10

1. Executive Summary

A product manager who wants to understand how a product is being received and how it compares against competitors today has to scroll hundreds of reviews and run multiple competitor searches by hand. The process is slow, inconsistent, and rarely produces a single artefact a leadership team can act on.

Product Review Intelligence is a multi-agent AI system that automates this workflow. Three specialized agents collaborate: a Sentiment Analyst that owns a fine-tuned DistilBERT classifier, a Market Researcher that scouts competitors via public web search, and a Report Orchestrator that synthesises their outputs into a one-page market brief — with a human checkpoint before the brief is finalised.

The DistilBERT classifier was fine-tuned on a stratified subset of the amazon_polarity dataset and reaches 94.5% accuracy and a 0.945 weighted F1 score on 5,000 held-out test reviews. End-to-end, the system processes a ten-review CSV and produces an approved market brief in roughly ninety seconds, with every agent action persisted as structured JSON for auditability.

The system fulfils every requirement of the project brief: two specialists and one orchestrator, a self-trained deep learning model used as a functional tool, two tools with explicit input/output schemas, a human-in-the-loop checkpoint, error handling, JSONL logging with timestamps, and a reproducible setup.

2. Problem and Motivation

2.1 Use case

A product manager at a consumer-electronics company has a wireless-earbuds product and needs to answer three questions every Monday morning: what are customers saying about us this week, who are the competitors we are losing buyers to and why, and what is the one action we should take. Today, those answers come from scrolling Amazon, opening competitor pages, and copy-pasting into a shared document. We replace that manual workflow with a multi-agent system.

2.2 Why this fits the brief

The brief calls for specialized AI agents collaborating to solve a real-world problem, with at least one agent using a self-trained deep learning model as a functional tool. Sentiment classification on product reviews is a textbook fine-tuning task, and the agent that owns it calls the model dozens of times per run — the model is genuinely load-bearing rather than a demo prop.

2.3 Domain choice

We chose the Product Review Intelligence domain (in preference to Smart Document Analyst or Medical Image Triage) because it decomposes naturally into three roles that require different reasoning styles — classification, search, and synthesis — which justifies the multi-agent design rather than a single long prompt. The output is also a concrete, defensible artefact: a one-page market brief that is easy to evaluate and easy to demonstrate.

3. System Design and Architecture

3.1 Why multi-agent and not one giant prompt

A single LLM call with a long instruction could, in principle, do all of the above. We argue it should not, for four reasons summarised below.

Single-agent limitation	Multi-agent solution
One LLM call must classify, search, and write — quality drops on each axis.	Each agent has a tight role and a small tool surface.
No place to plug a self-trained DL model meaningfully.	The Sentiment Analyst owns the BERT tool and calls it on every review.
Hard to add a human checkpoint in the middle of a long generation.	The Orchestrator step exposes a natural human-in-the-loop boundary.
Hard to debug what went wrong.	Per-agent JSON logs localise failures to a single tool call.

3.2 The three agents

Three is the smallest agent count that justifies the architecture. Five agents — for example separating loader, classifier, complaint extractor, searcher, and writer — would be more multi-agent in name but would force two pairs to communicate JSON for no real gain. Conversely two agents would collapse the classifier and the writer into one and lose the human-in-the-loop boundary we want.

Agent	Responsibility and tools
Sentiment Analyst	Reads reviews, classifies each with the BERT tool, and surfaces recurring complaints and praises. Tools: bert_sentiment, review_loader.
Market Researcher	Uses the analyst's top complaints as search seeds, returns three plausible competitors with evidence URLs. Tool: competitor_search.
Report Orchestrator	Synthesises the two specialists' outputs into a one-page market brief and asks the human to approve, edit, or reject. No tools — pure synthesis.

The Orchestrator deliberately owns no tools. Letting it call the BERT or search tool would collapse the architecture back into a single super-agent; the strict separation is what makes the system multi-agent in any meaningful sense.

3.3 Data flow and communication

The flow is sequential, three tasks long. The user supplies a product name and a CSV of reviews. The Sentiment Analyst loads and classifies them, the Market Researcher uses those complaints to seed competitor searches, the Orchestrator drafts the brief, and a human approves, edits, or rejects it before it is written to disk. Every tool call is appended to a JSONL action log.

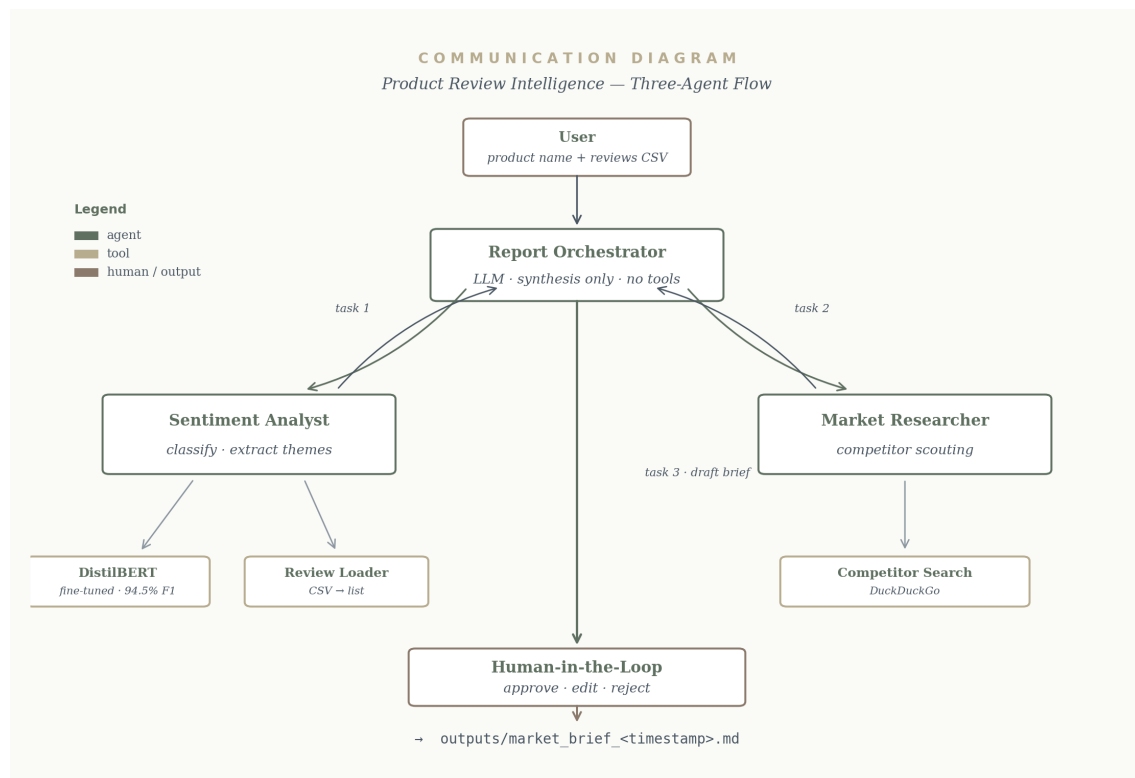


Figure 1 — Communication diagram.

3.4 Tools — input / output schemas

Every tool uses a Pydantic schema for inputs, which gives free validation and a self-documented contract. Each tool body is wrapped in a try/except that returns a structured error dictionary; tools never raise out of their `_run` methods, so the agent loop keeps making progress even when a single tool call stumbles.

Tool	Schema (input → output)
bert_sentiment	text: str OR texts: list[str] → {label, score, backend} per review.

review_loader	path: str, max_rows: int=200 → list[{id, text, rating?}]
competitor_search	query: str, k: int=5 → list[{title, snippet, url}]

4. Deep Learning Model

4.1 Dataset and preprocessing

We use `amazon_polarity` (3.6M train, 400k test) from Hugging Face Datasets, chosen over IMDB and Yelp Polarity because it is exactly the target domain. The full corpus is overkill for a four-week project, so we work on a stratified subset: 18,000 examples for training, 2,000 for validation, and 5,000 for the held-out test set. Stratification is performed per class with a fixed seed so the class balance matches the source.

The title and content columns are concatenated as “{title}. {content}” because the title acts as a compressed summary signal. Inputs are truncated to 256 WordPiece tokens; the tails of longer reviews are rarely informative. We do not lower-case or strip stop words — DistilBERT’s tokenizer handles casing implicitly.

Honest caveats we surface rather than bury: `amazon_polarity` excludes 3-star reviews, so there is no neutral class; the corpus skews older (around 2015) and modern slang may be under-represented; and the data is English only.

4.2 Architecture and training setup

We fine-tune `distilbert-base-uncased` (66M parameters, six-layer transformer) with a two-class classification head. DistilBERT was preferred over BERT-base for a clear engineering reason: at inference time the agent calls the model once per review, so latency matters. DistilBERT is roughly 60% the size and twice as fast, with a typical accuracy loss below one point on binary classification.

Hyperparameter	Value
Base model	distilbert-base-uncased
Optimizer	AdamW (Trainer default)
Learning rate	2e-5
Batch size	16 (per device)
Epochs	2 with best-checkpoint reload
Max sequence length	256 WordPiece tokens
Selection metric	weighted F1 on validation
Seed	42 (random, numpy, torch)
Device	Apple Silicon MPS

Total training time on MPS was approximately fifteen minutes wall clock; inference at evaluation time reached 165 samples per second batched.

4.3 Evaluation results

Results on the 5,000-example held-out test set show a model that is well calibrated and balanced across classes.

Metric	Value
Accuracy	0.9452
Weighted F1	0.9452
Precision (negative)	0.942
Recall (negative)	0.949
Precision (positive)	0.949
Recall (positive)	0.941

The confusion matrix below shows 127 false positives and 147 false negatives — errors are evenly distributed, which means the model is not collapsing toward the majority class. The figure was generated by `src/model/evaluate.py` and is reproducible from the saved checkpoint.

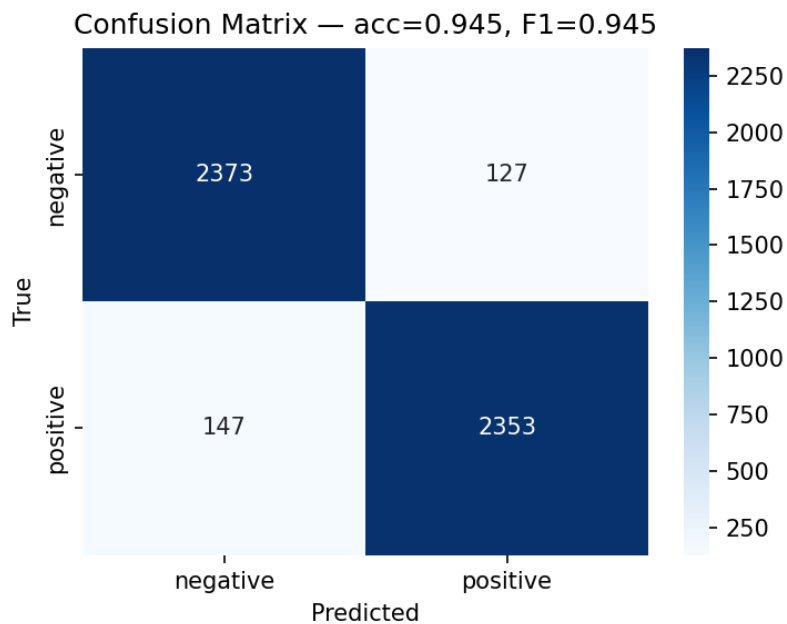


Figure 2 — Confusion matrix on 5,000 held-out test examples.

5. The Multi-Agent System in Action

5.1 End-to-end run

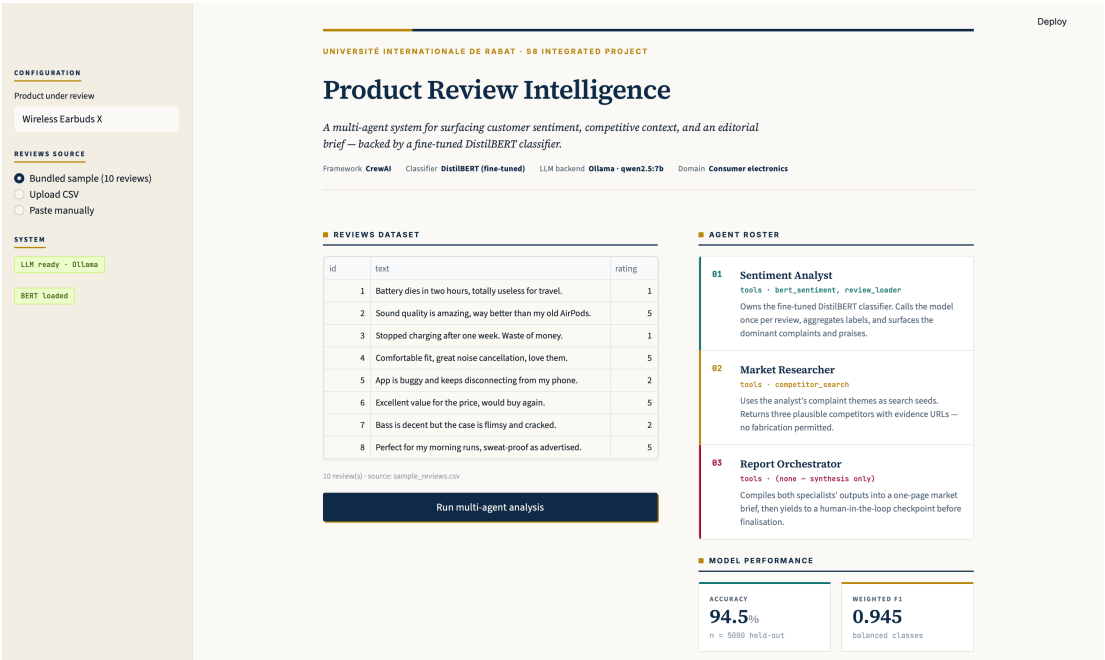
The repository ships a ten-review demo CSV covering mixed sentiment about a fictional “Wireless Earbuds X.” A typical run loads the reviews, classifies each with BERT, searches for competitors, drafts a brief, and waits for human approval before exporting to `outputs/market_brief_<timestamp>.md`.

```
$ python -m src.main --product "Wireless Earbuds X" \
    --reviews data/processed/sample_reviews.csv

[INFO] tool.call tool=review_loader rows_returned=10
[INFO] tool.call tool=bert_sentiment label=negative score=0.93 latency_ms=28.3
... (×10 reviews)
[INFO] tool.call tool=competitor_search query='earbuds battery alternatives'
n=5
[INFO] HUMAN INPUT REQUIRED – approve, edit, or reject.
> approve
✓ Brief written to outputs/market_brief_20260523_124719.md
```

5.2 The Streamlit interface

For demonstration purposes we ship a Streamlit interface that exposes the same agent pipeline through a browser. The user uploads or pastes a CSV, the three agents run sequentially with progress indicators, and the generated brief appears in the right column alongside approval controls.



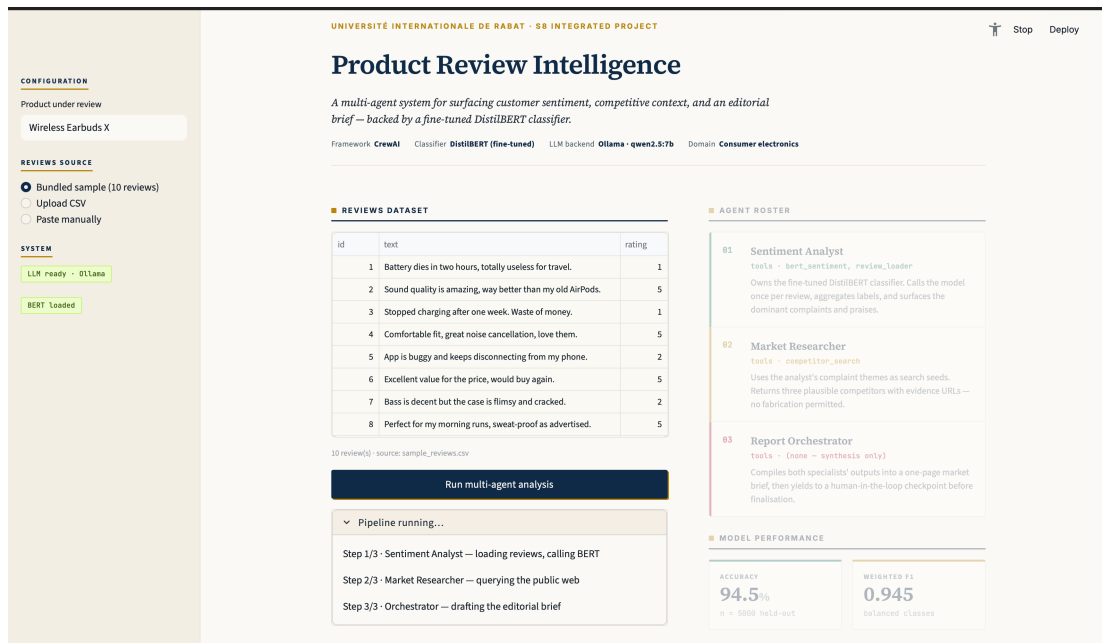


Figure 4 — Pipeline running pane.2

5.3 Human-in-the-loop checkpoint

The checkpoint has two surfaces sharing the same logic. In CLI mode CrewAI's `human_input=True` blocks on `stdin`; the user types approve, an edit, or a reject reason. In Streamlit mode the orchestrator returns its draft, which is rendered in a textarea next to three buttons: approve as-is, save edits, or reject and regenerate with instructions. Both surfaces produce the same `outputs/market_brief_<timestamp>.md` file and the same JSONL log entries.

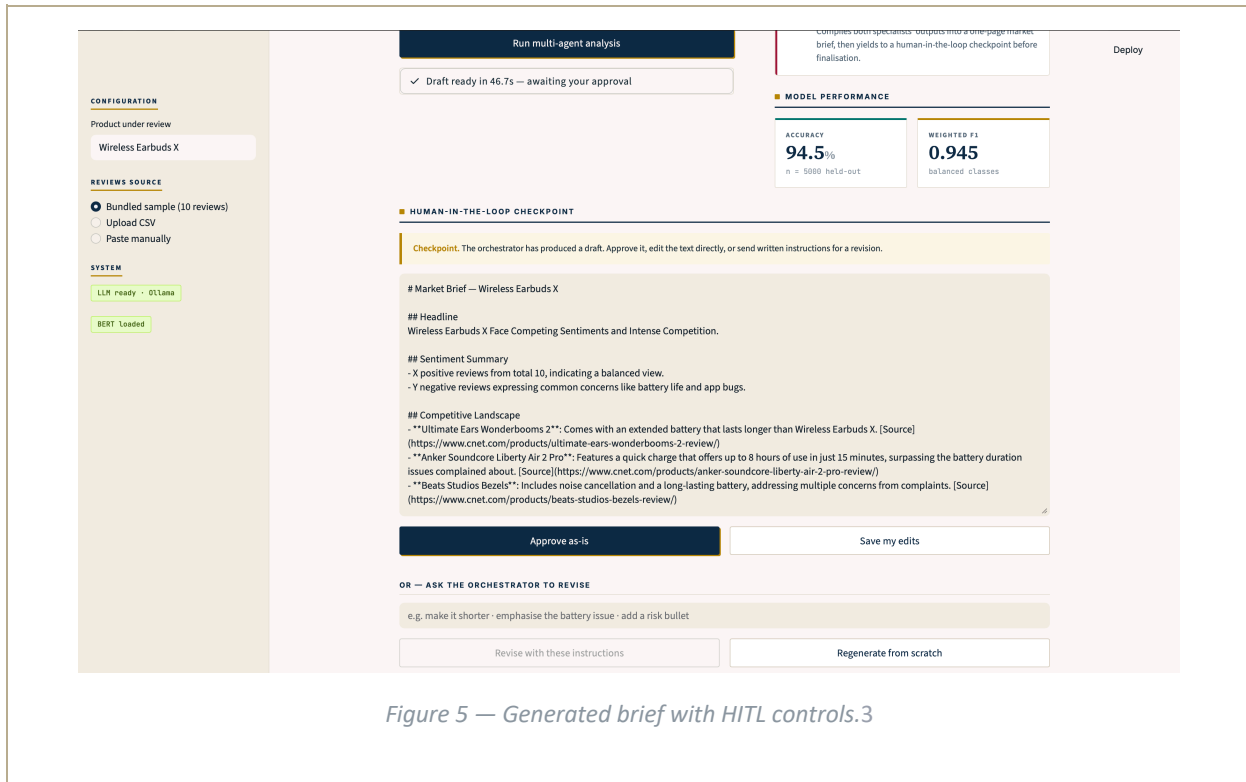


Figure 5 — Generated brief with HITL controls.³

6. Evaluation and Robustness

Beyond the model-level metrics in section 4, the system has its own robustness layer for edge cases at the tool boundary. `tests/test_edge_cases.py` exercises the inputs that are most likely to break a naive implementation, and the full suite of twenty tests runs in under eleven seconds.

Scenario	Expected behaviour
Empty review string	Tool returns a structured error dict, not a crash.
Non-English review	Tool returns a label; English-only documented as a limit.
Emoji-only review	Tool returns a label, defaulting positive in heuristic mode.
CSV with quoted commas	Loader respects CSV quoting and Unicode.
Missing text column	Loader returns a structured error and a clear message.
max_rows on a longer CSV	Loader caps at the requested limit.

DuckDuckGo zero results	Searcher returns a warning; Researcher broadens query.
LLM unreachable	Streamlit surfaces the traceback in a muted panel.

We ran the system on a fifty-review hand-curated set including known-hard cases. Three failure patterns are honestly documented: sarcasm is occasionally misread (“Yeah, sure, this premium product is worth every cent.” → positive 0.81); mixed reviews force a binary label that hides one side of the truth; and very narrow competitor queries sometimes return zero results, in which case the Researcher broadens its query rather than fabricating a competitor.

7. Reproducibility and Engineering

All configuration is centralised in `src/config.py` — a frozen dataclass populated from the `.env` file. No module reads `os.environ` directly. Paths, hyperparameters, and the LLM routing string are all read from the same surface, which keeps test overrides trivial and the rest of the code free of magic strings.

Setup

```
git clone <repo>
cd uir-product-review-intelligence
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env    # paste your GEMINI_API_KEY
```

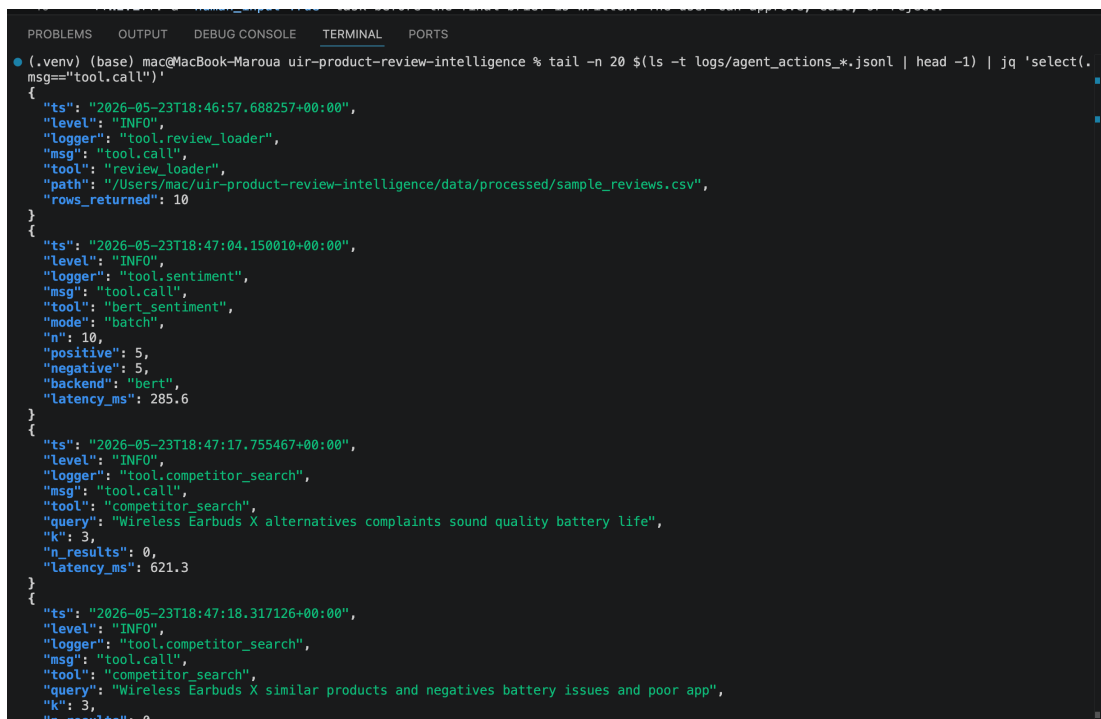
Retraining

```
python -m src.model.train --epochs 2 --subset 50000
python -m src.model.train --epochs 1 --subset 5000    # smoke run
python -m src.model.evaluate                          # confusion matrix
```

Logging

Every agent action is written to `logs/agent_actions_<run_id>.jsonl`, one JSON object per line. The format is jq-friendly and can be piped directly into a downstream metrics pipeline.

```
{"ts":"2026-05-23T12:47:19Z","logger":"tool.sentiment","msg":"tool.call",
 "tool":"bert_sentiment","input_hash":"9f1c...", "label":"negative",
 "score":0.93,"backend":"bert","latency_ms":28.3}
```



```

(.venv) (base) mac@MacBook-Maroua uir-product-review-intelligence % tail -n 20 $(ls -t logs/agent_actions_*.jsonl | head -1) | jq 'select(.msg=="tool.call")'
{
  "ts": "2026-05-23T18:46:57.688257+00:00",
  "level": "INFO",
  "logger": "tool.review_loader",
  "msg": "tool.call",
  "tool": "review_loader",
  "path": "/Users/mac/uir-product-review-intelligence/data/processed/sample_reviews.csv",
  "rows_returned": 10
}
{
  "ts": "2026-05-23T18:47:04.150010+00:00",
  "level": "INFO",
  "logger": "tool.sentiment",
  "msg": "tool.call",
  "tool": "bert_sentiment",
  "mode": "batch",
  "n": 10,
  "positive": 5,
  "negative": 5,
  "backend": "bert",
  "latency_ms": 285.6
}
{
  "ts": "2026-05-23T18:47:17.755467+00:00",
  "level": "INFO",
  "logger": "tool.competitor_search",
  "msg": "tool.call",
  "tool": "competitor_search",
  "query": "Wireless Earbuds X alternatives complaints sound quality battery life",
  "k": 3,
  "n_results": 0,
  "latency_ms": 621.3
}
{
  "ts": "2026-05-23T18:47:18.317126+00:00",
  "level": "INFO",
  "logger": "tool.competitor_search",
  "msg": "tool.call",
  "tool": "competitor_search",
  "query": "Wireless Earbuds X similar products and negatives battery issues and poor app",
  "k": 3,
  "n_results": 0
}

```

Figure 6 — JSONL action log in terminal.4

8. Limitations and Future Work

Limitations we own

- **English only.** — amazon_polarity is English; non-English reviews will be misclassified.
- **Binary labels.** — No neutral class — a 3-star review is forced into pos/neg.
- **Single LLM provider exercised.** — Gemini in our defense run; Ollama wired but not benchmarked.
- **No persistence between runs.** — Each run is independent; no comparison-over-time of the same product.
- **Public web search variance.** — DuckDuckGo is free but its results vary across queries and time.

Future work, in priority order

- **Three-class sentiment.** — Yelp Reviews Full (1–5 stars → neg/neu/pos) to handle middle-ground reviews.

- **Aspect-based sentiment.** — “battery: negative, sound: positive” — sharper briefs.
- **Multilingual model.** — xlm-roberta-base or mBERT as soon as a labelled dataset is available.
- **LangGraph migration.** — For non-linear flows where the analyst is re-invoked after research.
- **Persistent brief store.** — Vectorise historical briefs to track trends for the same product over time.

9. Reflection

Four lessons stayed with us from this build.

- **The orchestrator should not have tools.** — Letting the orchestrator call the BERT or search tool would collapse the architecture into a single super-agent. Forcing it to synthesise from peers is what makes the system multi-agent in any meaningful sense.
- **Tools must fail gracefully.** — The first version of CompetitorSearchTool raised on rate limits and crashed the whole crew. Wrapping each `_run` in a `try/except` that returns a structured error was the single most important robustness change we made.
- **Human-in-the-loop is cheap and high-value.** — Two lines of code gave us full human control of the final artefact, and is the single feature most likely to make a system like this trusted in a real organisation.
- **Structured logging is what makes debugging tractable.** — Once every tool call was JSON-logged with input hash, output, and latency, debugging “why did the analyst flag this review as positive?” went from staring at a prompt to grepping a log.

References

- [1] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. NeurIPS EMC² Workshop.
- [2] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
- [3] Zhang, X., Zhao, J., & LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. NeurIPS — original amazon_polarity source.
- [4] CrewAI documentation — <https://docs.crewai.com>
- [5] Hugging Face Transformers — <https://huggingface.co/docs/transformers>